

2026 Quantum Korea Hackathon Prompt

Trotterized Quantum Simulation

Abstract

Trotterization is one of the most widely used approaches to implementing Hamiltonian time evolution on a quantum computer. By decomposing the exponential of a Hamiltonian into a product of simpler exponentials, this practical method for quantum simulation exhibits a trade-off between approximation accuracy and circuit depth. Although this asymptotic trade-off is suboptimal compared with quantum singular value transformation, its simplicity at small scales makes it suitable for applications in the early fault-tolerant regime. In this challenge, participants will study Trotterized time evolution for a periodic Heisenberg model, analyze the resulting circuit structure, and investigate how spectral information can be extracted from the dynamics. Teams are encouraged to build on the official quantum simulation materials, replacing the tutorial Hamiltonian with the Heisenberg model below and exploring it creatively. The project may include simulation, circuit optimization, controlled time evolution, or spectral estimation based on autocorrelation signals.

Model

Consider the periodic anisotropic Heisenberg chain with a transverse X field on N qubits,

$$H = \sum_{j=0}^{N-1} (J_x X_j X_{j+1} + J_y Y_j Y_{j+1} + J_z Z_j Z_{j+1}) + h_x \sum_{j=0}^{N-1} X_j, \quad (1)$$

where $j+1$ is understood modulo N . Unless otherwise stated, use

$$N = 6, \quad J_x = 1.0, \quad J_y = 0.7, \quad J_z = 1.2, \quad h_x = 0.4. \quad (2)$$

Use the initial product state

$$|\psi_0\rangle = |010101\rangle, \quad (3)$$

where the bit string is written in the qubit-0-first convention used in the guide notebook. Use the time grid

$$t = 0, 0.05, 0.10, \dots, 15.0. \quad (4)$$

Equivalently, $\Delta t_{\text{grid}} = 0.05$ and $T_{\text{max}} = 15.0$.

Problems

Refer to the [guide notebook](#) for implementation and background details.

1. Basic Construction of Trotterized Time-Evolution Circuits

Reference: [IBM Quantum Learning: Utility-scale quantum computing – Quantum simulation](#).

Follow the reference material above and replace the tutorial Hamiltonian with the Hamiltonian defined in this prompt. The goal of this problem is to construct Trotterized approximations to e^{-iHt} , analyze their circuit structures, and compare their errors with exact classical simulation.

- (a) Construct the Hamiltonian using `SparsePauliOp`. Report the total number of Pauli terms and identify which terms come from the periodic boundary condition.
- (b) Construct product-formula circuits for each target time t in the time grid. For each t , directly approximate

$$U(t) = e^{-iHt} \quad (5)$$

using `PauliEvolutionGate` with the following synthesis options:

```
LieTrotter(reps=k),  
SuzukiTrotter(order=2, reps=k),  
SuzukiTrotter(order=4, reps=k).
```

Use $k = 1, 2, 3$.

- (c) Decompose or transpile each circuit into the basis gates

$$\{\text{cx}, \text{u3}, \text{u1}\}. \quad (6)$$

For each synthesis option and each repetition number, report the circuit depth, total gate count, number of `cx` gates, and number of one-qubit gates.

- (d) Use exact classical simulation for $N = 6$ to obtain the reference state

$$|\psi_{\text{exact}}(t)\rangle = e^{-iHt} |\psi_0\rangle. \quad (7)$$

Compare it with the product-formula state $|\psi_{\text{PF}}(t)\rangle$ using the state infidelity

$$\epsilon_{\text{state}}(t) = 1 - |\langle \psi_{\text{exact}}(t) | \psi_{\text{PF}}(t) \rangle|^2. \quad (8)$$

- (e) Compute local-observable errors for

$$Z_1, \quad Z_0 Z_1. \quad (9)$$

For an observable O , use

$$\epsilon_O(t) = |\langle O \rangle_{\text{exact}}(t) - \langle O \rangle_{\text{PF}}(t)|. \quad (10)$$

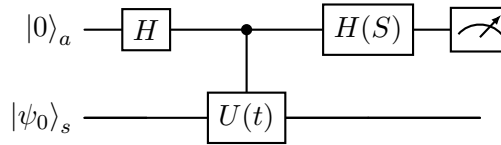
- (f) Plot time versus error for the state infidelity, Z_1 error, and $Z_0 Z_1$ error. Compare Lie–Trotter, second-order Suzuki–Trotter, and fourth-order Suzuki–Trotter formulas.
- (g) Discuss the trade-off between approximation accuracy and circuit depth. Which construction is best for the same repetition number? Which construction is best for a similar circuit depth?
- (h) Run a small subset of the circuits on an available IBM Quantum backend. Choose a few representative time points from the time grid and estimate $\langle Z_1 \rangle$ and $\langle Z_0 Z_1 \rangle$ using one product-formula construction. Use Qiskit Runtime error mitigation, for example by setting `resilience_level=2`, and compare the backend results with the ideal exact-simulation values. Discuss the main sources of discrepancy.

2. Spectral Information from Autocorrelation Signals

In this problem, study the autocorrelation signal

$$A(t) = \langle \psi_0 | e^{-iHt} | \psi_0 \rangle. \quad (11)$$

- (a) Compute the exact autocorrelation signal $A_{\text{exact}}(t)$ on the time grid using classical simulation. Plot its real part, imaginary part, and absolute value.
- (b) Estimate the Trotterized autocorrelation signal $A_{\text{PF}}(t)$ using at least one product-formula setting from Problem 1 by constructing Hadamard circuits:



A simple option is to compute the overlap between the initial state and the Trotterized final state in statevector simulation.

- (c) Compare $A_{\text{PF}}(t)$ with $A_{\text{exact}}(t)$. Plot the autocorrelation error

$$\epsilon_A(t) = |A_{\text{exact}}(t) - A_{\text{PF}}(t)|. \quad (12)$$

- (d) Apply a discrete Fourier transform to the exact autocorrelation signal. Plot the resulting spectrum and identify the dominant peaks.
- (e) Explain what spectral information is contained in $A(t)$. In particular, discuss why the peak locations are related to eigenenergies that have nonzero overlap with the initial state $|\psi_0\rangle$.

Bonus Problem 3. Simple Circuit Optimization

Reference: [IBM Quantum Tutorial: Compilation methods for Hamiltonian simulation circuits, Part 2.](#)

This is a bonus engineering problem. Use one representative circuit, for example `SuzukiTrotter(order=2, reps=3)` at $t = 0.5$.

- (a) Transpile the circuit for one IBM Quantum backend or fake backend. Compare optimization levels 0, 1, 2, 3.
- (b) Report only the essential resource counts: circuit depth, total gate count, and two-qubit gate count.
- (c) If available in your environment, try Rustiq-based synthesis for `PauliEvolutionGate` and compare it with the standard transpilation result.

References

1. IBM Quantum Learning, [“Utility-scale quantum computing: Quantum simulation.”](#)
2. IBM Quantum Documentation, [“Compilation methods for Hamiltonian simulation circuits.”](#)
3. Y. Thimithree et al., [“Faster and Shorter Synthesis of Hamiltonian Simulation Circuits,”](#) arXiv:2404.03280.